CSS 430 Operating Systems
Program 1B: Process Management – Piped Processes

1. Purpose

This assignment is designed to ensemble an execution of descendent processes with pipe-based communication. We will use *fork, execlp, pipe*, *dup2*, and *wait,* each spawning a child process, running a given program within the child, creating a pipe between parent and child processes as an inter-process communication channel, remapping the pipe to a different file descriptor, and waiting for the child to finish.

2. Linux System Calls

The following table summarizes Linux system functions used in program 1B.

Linux System Functions	Actions
pid_t fork( )	Creates a child process. The parent process receives the child process ID, while the child receives 0 as a return value. The child starts right after fork( ).
int execlp(const char *path, const char *arg, ...)	Overwrites the calling process image with a new program located at path and starts its execution from main( ) as passing arg to it. No return on success.
int pipe(int pipefd[2])	Creates a pipe, a unidirectional data channel. pipefd[0] refers to the read end of the pipe, whereas pipefd[1] referr to write end of the pipe. Zero is returned on success.
int dup2(int oldfd, int newfd)	Makes newfd be the copy of oldfd. This is frequently used to map a pipe's read/write ends to standard input/output. For example, dup(pipefd[0], 0) and dup(pipfd[1], 1).
int close(int fd)	Close a file descriptor. Use it for closing unnecessary pipe ends.
pid_t wait(int *status);	Waits for a child process to complete its execution and returns the child process ID. The parent receives in status the return value from the child's main( ) or exit( ).

3. I/O Redirection and Pipeline

One of the *shell*'s interesting features is I/O redirection.

```
cssmpi1h$ a.out < file1 > file2
```

redirects *a.out*'s standard input and output to *file1* and *file2* respectively, so that *a.out* reads from *file1* and writes to *file2* as if it were reading from a keyboard and printing out to a monitor. Another convenience feature is pipeline.

```
cssmpi1h$ command1 | command2 | command3
```

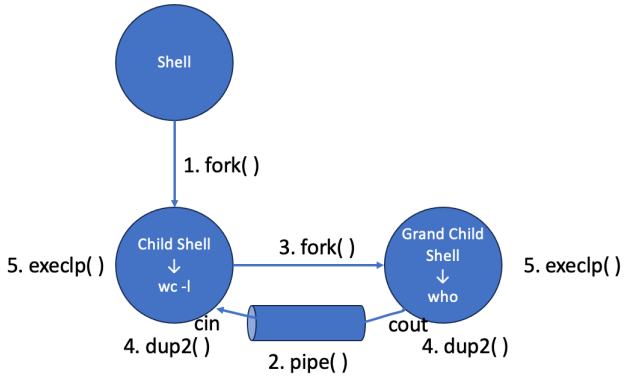
connects *command1*'s standard output to *command2*'s standard input, and also connects *command2*'s standard output to *command3*'s standard input. For instance,

```
cssmpi1h$ who | wc -l
```

First executes the *who* command that prints out a list of current users. This output is not displayed but is rather passed to *wc*'s standard input. Thereafter, the *wc* is executed with its *-l* option. It reads the list of current users and prints out #lines of this list to the standard output. As a result, you will get the number of the current users.

4. Shell Implementation Techniques

If the *shell* receives a sequence of commands, each concatenated with "|", it must recursively create children whose number is the same as that of commands. Which child executes which command is a kind of tricky. The farthest offspring will execute the first command, the grand child will execute the 2nd last, and the child will execute the last command. Their standard input and output must be indirected accordingly using the **pipe** and **dup2** system calls. The following pictures describe how to execute "who | wc -l", and how to use the **pipe** system call.



5. Statement of Work

Task 1: Complete *processes.cpp* you can find under ~css430/prog/1B/. This program receives one argument, (i.e., argv[1]) upon its invocation and searches how many processes whose name is given in argv[1] are running on the system where your program has been invoked. To be specific, your program should demonstrate the same behavior as:

```
ps -A | grep argv[1] | wc -l
```

Implement *processes* using the system functions listed above in Section 2: Linux System Calls.

For more details, type the **man** command from the *shell* prompt line. Use only the system calls listed above. Do not use the **system** system call. Imitate how the *shell* performs "ps -A | grep argv[1] | wc -l". In other words, your parent process spawns a child that spawns a grand-child that spawns a great-grand-child. Each process should execute a different command as follows:

Process	Command	Standard in (fd 0)	Standard out (fd 1)
Parent: starts main( )	wait for a child to complete	no change	no change
Child	wc -l	dup2(the read end of the pipe shared with grand child, 0)	no change
Grand child	grep argv[1]	dup2(the read end of the pipe shared with grand-grand child, 0)	dup2(the write end of the pipe shared with child, 1)
Grand-grand child	ps -A	no change	dup2(the write end of the pipe shared with grand child, 1)

To verify the correctness of your processes.cpp, compare the following two executions:

```
[css430@cssmpi1 1B]$ ps -A | grep ssh | wc -l
3
[css430@cssmpi1 1B]$ g++ processes.cpp
[css430@cssmpi1 1B]$ ./a.out ssh
3
commands completed
[css430@cssmpi1 1B]$
```

The lines of code (LoC) for this implementation would be less than 40 lines (excluding comments).

Task 2: Illustrate all these processes and their pipe connections involved in processes.cpp.

6. What to Turn in

	Materials	Points
1	processes.cpp a. Code organization: **+2pts** • Well organized: 2pts, • Poor comments or bad organization: 1pt, or • No comments and horrible code: 0pts b. Correctness: **+10pts** • A parent waits for a child: **+1pt,** • A child executes "wc -l": **+3pts,** • A grand-child executes "grep argv[1]": **+3pts, and** • A great-grand-child executes "ps -A": **+3pts**	12
2	An execution snapshot in txt, jpg, png, or pdf • The same output as ps -A | grep argv[1] | wc -l: 4pts, • The different output from ps -A | grep argv[1] | wc -l: 2pts, or • No outputs: 0pts	4
3	An illustration of all processes and their pipe connections involved in processes.cpp • Correct: 4pts • Wrong: 2pts, or • No illustrations: 0pts	4

Total 20pts.